

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED
ELETTRICA E MATEMATICA APPLICATA



Computer Engineering
Artificial Vision

Pedestrian attribute recognition competition

Professors:

Prof. Mario Vento

Prof. Antonio Greco

Students:

Carmine Aliberti

Giovanni Casella

Christian Salvatore De Angelis

Nunzio Del Gaudio

YEAR 2024/2025

Contents

1	Introduction	2
2	Trajectory	4
2.1	Point mapping	5
2.1.1	Using PPM	5
2.1.2	Projection Matrices	5
2.2	Line Detection	6
3	Detection and Tracking implementations	7
3.1	Detection	7
3.2	Tracking	8
4	Person Attribute Recognition	10
4.1	Dataset	10
4.1.1	Data Reorder	11
4.1.2	Preprocessing and Augmentation	11
4.2	Model	12
4.3	Loss Function	13
4.4	Training	14
4.5	Learning Plots	14
5	Final evaluation	19
5.1	Video Test	19
5.2	Possible Improvements	20

Chapter 1

Introduction

Person recognition and tracking in complex environments are essential tasks in various application domains, including video surveillance, human behavior analysis, and security management. This project aims to develop an advanced system for tracking individuals, accurately identifying their attributes, and storing this information in a database. The system will leverage computer vision and machine learning techniques.

This project aims to develop an advanced system that focuses on:

1. **Mapping image coordinates to real-world coordinates:** Implementing a module capable of accurately projecting points from real-world locations to image.
2. **Analyzing trajectories:** Evaluating the imaginary lines that each person crosses, providing insight into their movement patterns and behavior.
3. **Tracking individuals:** Follow people through video sequences in real-time.
4. **Identifying attributes:** Correctly identifying key attributes of the tracked individuals, such as:
 - Gender;
 - Whether they are wearing a hat;
 - Whether they are carrying a bag (e.g., backpack, briefcase);
5. **Optimization :** Ensure optimal real-time performance.

The system will be installed in the atrium of the University of Salerno, where the location is fixed. However, **the coordinates and direction of the crossing lines will be determined shortly before the software is activated as the position and the yaw, pitch, roll of the camera.**

The ultimate goal is to develop a high-precision real-time solution that can handle video data from this environment, ensuring accurate tracking, effective attribute identification, and storing relevant data in a database.

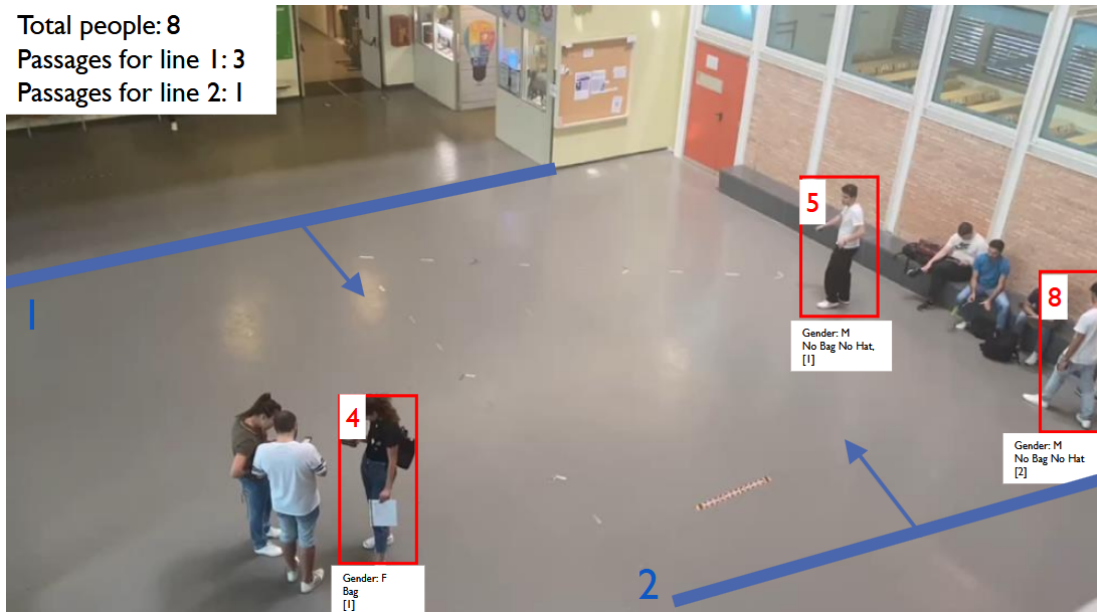


Figure 1.1: software output example.

Chapter 2

Trajectory

In this project, trajectory refers to the order in which a person crosses the imaginary lines drawn in the real world. These lines are not just spatial markers but also define a direction of crossing. A line is considered crossed only if it is crossed in the expected direction. Before conducting any trajectory analysis, it is necessary to develop a module that can draw these imaginary lines within the image captured by the camera.

To draw the lines, several points will be provided before the software is initialized.

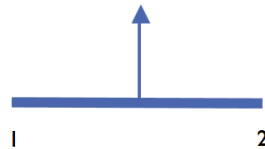


Figure 2.1: trajectory line example.

Each line is simply the segment connecting two of these points, with its direction determined by the order in which the points are given.

Therefore, the first step is to implement a submodule that can plot points on the image. However, these points are provided in a real-world Cartesian coordinate system with x , y , and z values. The submodule needs to map these 3D coordinates into the 2D image space, which involves projecting the real-world coordinates onto the camera's image plane.

So the steps are:

1. Projection of the point from real coordinates to image coordinates
2. The next step is to generate a segment connecting the two points provided, forming the crossing line.
3. The direction of the crossing line will be determined by the order in which the two points are provided.
4. Once the line and direction are established, the system will need to detect when a person crosses the line
5. The counting of crossings will be based on identifying when a person crosses from one side of the line to the other.
6. This will provide valuable data for analyzing pedestrian movement and behavior patterns.

2.1 Point mapping

The development of the point mapping module began with the goal of solving the specific problem of mapping points from the real world (with known coordinates x, y, z) to points in the image plane (u, v). From the outset, we are aware of the reference system x, y, z , the camera coordinates, and the point coordinates within this reference system. The process of mapping involves translating the real-world coordinates into the 2D image space by considering the camera's intrinsic and extrinsic parameters. This step is essential for ensuring that any points of interest in the real world can be properly located in the camera's view, laying the foundation for accurate analysis and tracking in later stages of the project.

2.1.1 Using PPM

We evaluated several options to achieve this goal. Initially, we considered implementing a module that would leverage the camera's field of view (FOV), the distance from the camera to the point, and the camera's resolution to convert the given coordinates in meters into pixel quantities. While this approach seemed straightforward, it proved to be inaccurate. The method failed to properly account for the increase in distance for each point on the floor, leading to significant errors in mapping.

Moreover, this solution became ineffective in the case of camera rotation, as it did not factor in the yaw and pitch of the camera, which are critical for accurate mapping when the camera orientation changes. This limitation highlighted the need for a more robust solution that could dynamically handle varying distances and camera rotations to achieve more precise point mapping.

2.1.2 Projection Matrices

Projection matrices are a very solid solution, as they account for camera rotation and are applicable in any context. Under this solution, there are several sub-solutions for coordinate conversion:

1. **From the real world to the camera:** This process essentially shifts the origin of the coordinate system to the camera's location, making the camera the reference point. The goal is to align the world coordinates with the camera's viewpoint, simplifying the subsequent projection process.
2. **From the camera to the image plane:** This step involves transforming the coordinate system from 3D to 2D, effectively removing the depth dimension. In this process, the 3D coordinates of a point in the camera's view are projected onto the 2D image plane, essentially flattening the 3D scene into a 2D representation. This transformation is achieved by using the camera's intrinsic parameters, which allow the depth information to be discarded while retaining the relevant 2D image coordinates for each point.
3. **Transforming spatial coordinates into pixel values:** In this step, the 2D coordinates obtained from the previous transformation are converted into pixel values on the image. This involves using the camera's intrinsic parameters, specifically the focal length and sensor size, to map the normalized 2D coordinates onto the actual pixel grid of the camera's image. The result is the position of the point in the image space, expressed in terms of pixel coordinates (u, v), allowing the point to be accurately located on the camera's captured image.

The transformation methodology assumes that the z-axis represents "depth" and the y-axis represents "height", while the real-world coordinate system provides the coordinates with y as depth and z as "height". Due to this difference in axis definitions, these two quantities are swapped in the script to ensure that the transformation correctly maps the real-world coordinates to the camera's coordinate system. This adjustment is necessary to maintain consistency between the real-world data and the image processing system, as the two systems use different conventions for axis orientation.

2.2 Line Detection

To create the lines, we connect the plotted points. Each line has a corresponding line that defines its direction, obtained by taking the perpendicular to the original line. To determine whether an object crosses a line, the process involves two steps:

1. Check if the line was crossed. The chosen approach is done by comparing the current bounding box with the one from the previous frame. The line connecting the feet of the two bounding boxes is analyzed to see if it intersects with the drawn line. If an intersection occurs, it indicates that the line was crossed. Initially, the center of the bounding box was used to determine whether the line was crossed. However, this approach presented issues with unexpected direction switches, as the line was sometimes marked as crossed even when the entity had not actually crossed it. The decision was so to choose the first solution.
2. Verifying the crossing direction. The trajectory is added only if the crossing direction matches the expected one. This is determined by computing the dot product between the bounding box line (computed in the previous step) and the directional line. If the dot product is greater than 0, the crossing direction is correct, and the trajectory is associated with the individual. Otherwise, it is ignored.

Chapter 3

Detection and Tracking implementations

3.1 Detection

To overcome the detection challenge, it was decided to use YOLOV8 with a focus on the "yolov8m" model. The "m" stands for medium and provides a good balance between speed and accuracy compared to the less accurate yet faster versions "s" and "n" or the larger "l" and "x" which are more accurate but slower. Various evaluations of YOLOv8 parameters were conducted to achieve a good balance between performance and accuracy. The following list will outline some iterations:

1. In the first iteration, the image resolution was set to 640x320, with a vid_stride of 15. The IoU was set to 0.3 and the confidence threshold (conf) to 0.6. During this iteration, the detection of individuals was suboptimal due to the low IoU, which led to the merging of some bounding boxes. Additionally, the low resolution resulted in missed detections, particularly in darker areas and the higher confidence threshold caused the miss detection of some people. The only advantage for this iteration was the execution performance which was good (*15ms/frame*).
2. The image resolution was set to 1280x720 with a vid_stride of 15. The IoU was set to 0.6 and the confidence threshold to 0.4. By increasing the IoU, the detection of individuals improved, but with a trade-off in performance and setting the Persist parameter to true was aimed at addressing the issues of ID switching. The high vid_stride value led to the loss of some detected individuals, in particular most of the problems were related to darker areas and groups of people (*30ms/frame*).
3. In another iteration the image resolution was set to 1920x1080 (the original resolution) with a vid_stride of 9. The IoU was kept at 0.6, and the confidence threshold (conf) was set to 0.4. Additionally, the maximum number of detected individuals was limited to 30 by adjusting the max_det parameter, which contributed to improved performance. It was noticed that the detection worked way better performance wise. (*45ms/frame*)

In the final test, the image resolution remained at 1920x1080, which is the original resolution of the video, and the vid_stride was unchanged initially. The IoU threshold was increased to 0.8, and the confidence threshold was reduced to 0.3. Enabling the half parameter improved execution speed but slightly reduced accuracy. Among the problems related to detection, the dark area problems were resolved by increasing the

image resolution. To resolve the problem related to groups of people, in which occlusion were quite frequent, noticing that increasing the IoU didn't get more result above 0.5, it was decided to lower the `vid_stride` to 7. Results on the testing video showed good performance except with one occlusion in the darker area, which caused the detection to start some frames later.

A possible tested solution was to increase the video brightness, but this lead to nowhere.

3.2 Tracking

For the tracking phase, the BoT-SORT tracker was utilized. This tracker was selected due to its strong ability to minimize ID switches considering the similarity in appearance and predicted position. Since the camera in this setup is in a fixed position, BoT-SORT performance is not hindered by complex camera motions. Due to this reason, it was decided to deactivate **gmc**, to gain performance. The initial parameters were modified after some adjustments, and various tests were conducted. Some of these tests showed these results:

1. Setting a high threshold of 0.9 and a low threshold of 0.55 resulted in too few tracked entities, sometimes even none.
2. Reducing the low threshold and lowering the high threshold helped to track all the entities. The low threshold was set to a very low value so that a lower number of detections had to be discarded to do the matching.
3. We setted the buffer size to 20 in order to avoid id switch when a person exits from the same point of the scene where another person enters.
4. Increasing the match threshold value significantly enhanced tracking accuracy . This value was set initially at 0.65 to have a match between people in dark area, however, by balancing the other parameters and increasing this to 0.85, it was possible to not lose the matches in darker area while also reducing the number of ID switch cases.
5. **with_reid** was left at false value because the re-identification wasn't the goal.
6. Setting a **new_track_threshold** high helped to improve the capability to retain existing tracks while limiting the creation of new tracks, which were frequent when occlusion occurred for some frames or when detector lost the detections.
7. **proximity threshold** was left at default value because it was possible to resolve occlusion and ID switch, by modifying `new_track_threshold` and `match_thresh`.

By discovering these it was possible to define the final parameters as follow:

- **proximity_threshold=0.5**
- **match_threshold=0.85**
- **new_track_threshold=0.85**
- **track_buffer=20**
- **track_high_threshold=0.40**

- `track_low_threshold=0.05`
- `with_reid=false`

The main challenges encountered during this process included: ID switching and the creation of new tracks in darker areas or during occlusions, as well as sudden direction changes that led to new ID assignments.

To address occlusion-related issues, the parameters were adjusted (e.g., modifying the matching threshold and the track threshold).

The most time-consuming issue was the assignment of new IDs during sudden direction changes. This was effectively resolved by combining the detector and tracker. If the detector could reliably track the object even during abrupt direction changes, the tracker would maintain the correct ID. This was achieved by reducing the frames per second using the `vid_stride` parameter. Otherwise, if the detector lost the object before the direction change (often accompanied by occlusion), the tracker would also fail to maintain the track.

Some issues remained unresolved, for example by reducing the resolution (compared to the original video resolution) , the tracker would not perform with maximum accuracy. This resulted in cases of ID switching or the creation of new tracks, likely because the detector lost more objects due to the lower resolution. It was decided to revert to the original video resolution, which offers excellent performance at the cost of increased execution time.

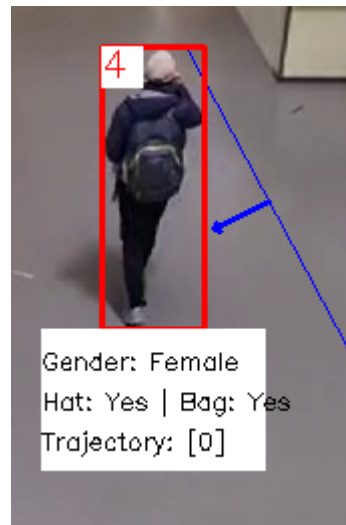


Figure 3.1: Example of Detection and Tracking.

Chapter 4

Person Attribute Recognition

In this section, we will analyze the pedestrian recognition module, specifically the creation of the classification network trained using a dataset of images provided by MIVIA.

4.1 Dataset

This section is necessary to document all the adjustments made to the dataset provided to us. By "dataset" we refer to 92,160 images in the training set and 12,161 images in the validation set. Each image has three labels (gender, bag, and hat), but not all labels are assigned a value (NaN values). Given the absence of a test set, it was decided to use the provided validation set as the test set and to use 20% of the training set images as the validation set.

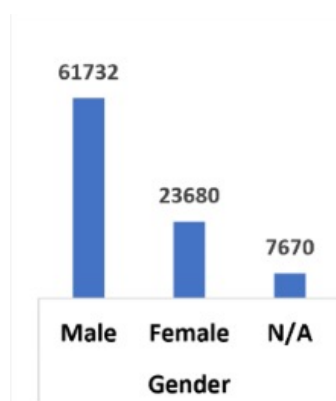


Figure 4.1: Gender Distribution.

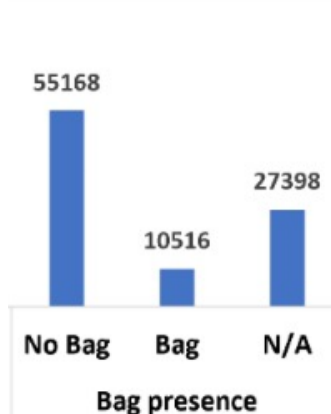


Figure 4.2: Bag Distribution.

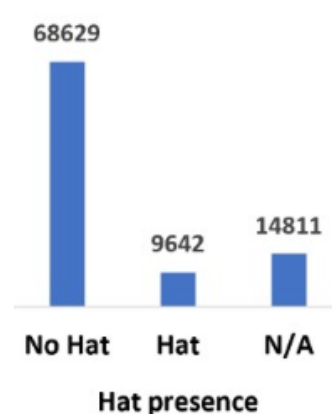


Figure 4.3: Hat Distribution.

- Gender values are [0,1] represent male and female, respectively.
- Bag values are [0,1] indicate whether the person is wearing the accessory (1) or not (0).
- Hat values are [0,1] indicate whether the person is wearing the accessory (1) or not (0).

4.1.1 Data Reorder

First of all, the fundamental modifications to the dataset were:

1. Removal of images with NaN values for all three labels (effectively useless for our purpose)
2. Application of a shuffle to the dataset.
3. Creation of batches such that for each feature there is at least one element different from the NaN value (in a batch, no feature contains only -1 values)

4.1.2 Preprocessing and Augmentation

Although most of the images had a similar aspect ratio, several transformations were applied to enhance model robustness and generalization before sending the images to the network. These transformations include:

- **Resize:** The images were resized to a consistent 224x224 resolution, which is the input size expected by the network, as it was trained with this specific configuration.
- **ColorJitter:** This transformation was applied to introduce color variations, enhancing the model's ability to generalize across different lighting and color conditions.
- **RandomHorizontalFlip:** Random horizontal flipping was applied to the images to increase the diversity of the dataset and improve the model robustness to orientation.
- **Histogram equalization (*custom*):** In order to make the images more independent of lighting conditions, we applied histogram equalization to the images.
- **ToTensor:** This transformation converts the image into a tensor format, which is necessary for input to the neural network.
- **Normalize:** This normalization setting matches the configuration used during the training of the network. The normalization values were adjusted to the standard ResNet50 settings, with **mean** = [0.485, 0.456, 0.406] and **std** = [0.229, 0.224, 0.225].

The resize and normalization are applied to better align with the backbone used (which was trained on these values).



4.2 Model

In this chapter the structure of the network will be presented.

From the beginning, we decided to use a multi-task network, specifically for binary classification tasks.

The first backbone tested was ResNet-18, which delivered poor results and an under-performing classifier. Subsequently, ResNet-34 was tested, yielding significantly better results compared to ResNet-18, albeit with a slight increase in computational requirements. While ResNet-34 demonstrated a good F1-score during evaluation on the test set, its performance on video testing was suboptimal.

Ultimately, ResNet-50 was chosen due to its superior accuracy, despite its higher computational demands. With its 50 layers, ResNet-50 is capable of learning complex features, making it well-suited for various vision tasks such as object detection. The model processes input images by extracting feature maps. Although the F1-scores for ResNet-34 and ResNet-50 were comparable, ResNet-50 outperformed ResNet-34 significantly in video-based testing, solidifying its selection as the preferred backbone.

The last layer of the backbone was replaced with a fully connected layer designed to **project** the data by reducing its dimensionality. Placing this layer before the attention layers allows it to fully leverage multi-task learning

Following the core architecture, the attention mechanism was applied to enhance the model’s capability to focus on different parts of the input sequence simultaneously **MultiHeadAttention**. We also tested the **CBAM** attention layer, but the performance in the video was not satisfactory.

Those layers allows the model to capture diverse patterns or dependencies by learning from multiple attention heads, each focusing on different aspects of the input, which are in this case gender, bag and hat.

Dropout has also been implemented in the fully connected layers to prevent under-utilization of the neurons in the layer. Graphically, the architecture can be represented as follows:

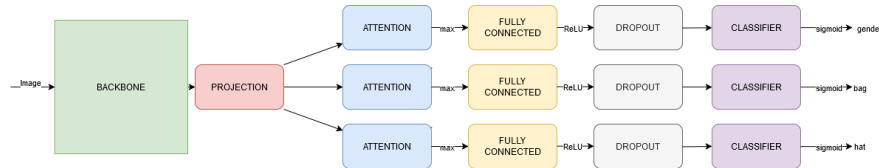


Figure 4.4: Network Architecture.

The decision to adopt a multitask approach was driven by the fact that using a single multitask model helps alleviate the computational complexity associated with training, testing, and real-time usage, which would otherwise require three separate single-task models as well as ease of training.

4.3 Loss Function

The loss function used to train the network was **BCEWithLogitsLoss**, a widely used choice for binary classification tasks. This loss function combines the sigmoid activation with the Binary Cross-Entropy (BCE) loss, which computes the loss between the probabilities and the true labels. By integrating the sigmoid activation into the loss function, **BCEWithLogitsLoss** eliminates the need to apply the sigmoid separately.

- The loss for NaN values is set as the average loss of the other losses for the same feature. This approach allows us to avoid removing data from the dataset while preventing errors during learning.
- The loss is weighted differently for positive and negative values based on the dataset distribution. If a feature has significantly more positive elements than negative ones, the loss for negative elements will have a higher value. This is done to balance the unrealistic distribution of the dataset and prevent introducing bias into the model.
- A higher weight was assigned to the bag classification loss as it demonstrated the lowest F1-score and posed the greatest challenge for classification. This adjustment ensured the model concentrated more on improving this specific aspect. The weighted loss strategy was implemented to prioritize the areas with weaker performance, driving the model to enhance its accuracy where it was most needed.

For the final model, static weights were used because an attempt with dynamic weights using **GradNorm** did not yield good results on the training set, as the F1-Score never exceeded a value of 0.6. All these adjustments are implemented by the adjusted loss function.

$$total_loss = (0.3 * loss_gender) + (0.2 * loss_hat) + (0.5 * loss_bag)$$

The learning rate was adjusted multiple times during the experimentation. A higher value led the model to overfit quickly, so ultimately, a learning rate of 0.00001 was chosen to achieve better stability and performance. Training was conducted over 15 epochs

4.4 Training

Adam was chosen as the optimizer, and **CyclicLR** as the scheduler, which gradually increase the learning rate to the chosen max value as the epochs progress, and then reduce it back to the initial learning rate.

Several training sessions were conducted using different parameters and network structures.

Various learning were tested. The most noteworthy observations include:

1. A learning attempt without training the backbone was made, but the results were quite disappointing. Therefore, every subsequent attempt was carried out with the backbone learning enabled, allowing for slight fine-tuning of the network.
2. The most unique learning experiment involved training the network with grayscale images. The rationale behind this attempt was the consideration that many of the elements to be classified could be recognized without color information. However, the idea was set aside after realizing that colors could be a valuable feature for detecting hats and bags.
3. Another experiment involved training without using histogram equalization. While the results on the test set were satisfactory, the performance on video (with or without CLAHE applied) was worse.
4. A learning attempt was made by halving the number of neurons in the classification heads, which resulted in an unacceptable drop in performance.

4.5 Learning Plots

Below, we train some metrics calculated on the validation set during learning.

As mentioned earlier, the network quickly achieves good results. To avoid overfitting, we decided to use the model obtained at the sixth epoch.

These tests were conducted by applying the same transformations to the validation images as those applied during training.

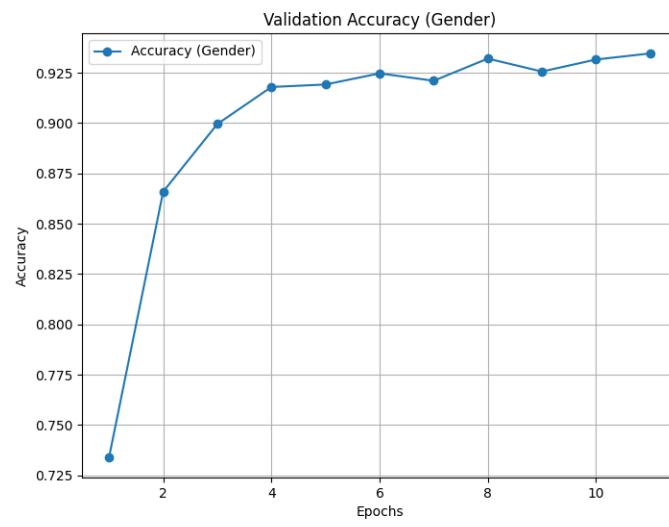


Figure 4.5

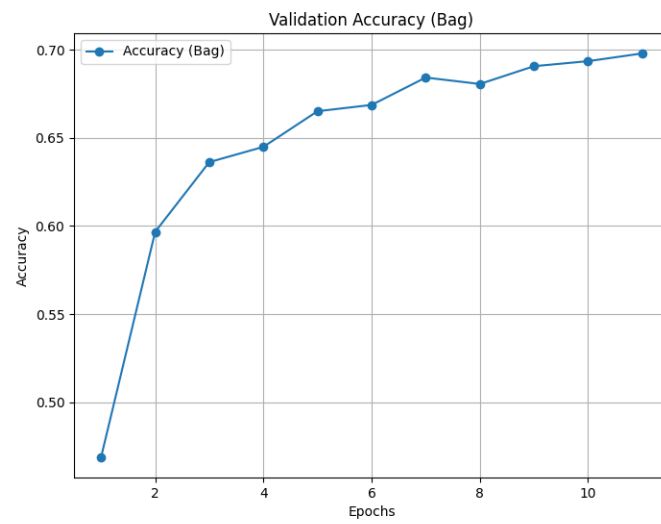


Figure 4.6

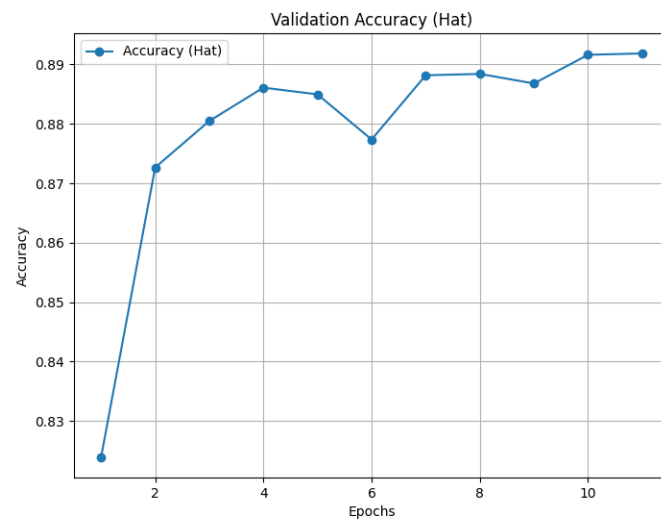


Figure 4.7

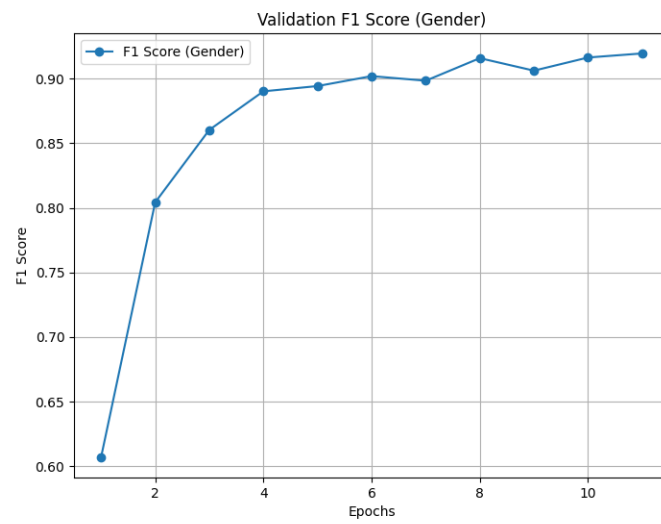


Figure 4.8

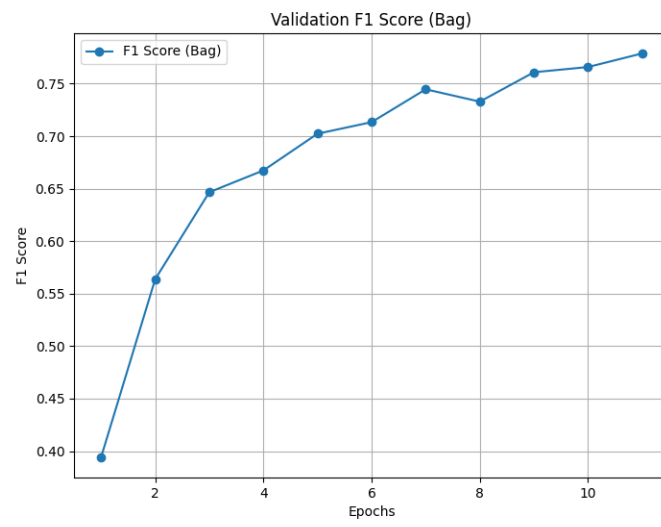


Figure 4.9

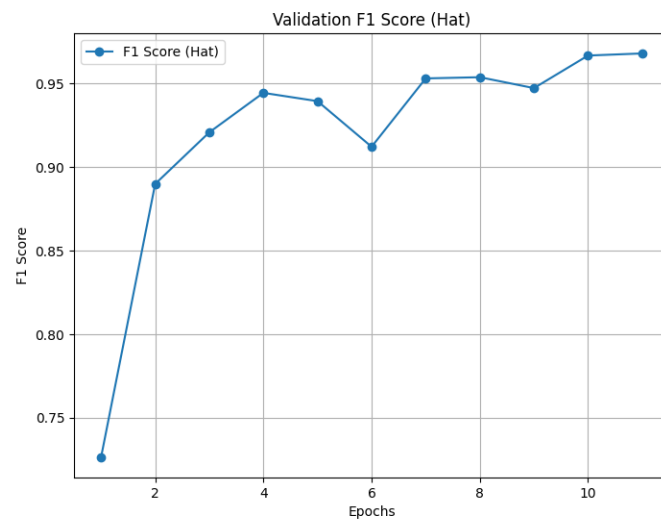


Figure 4.10

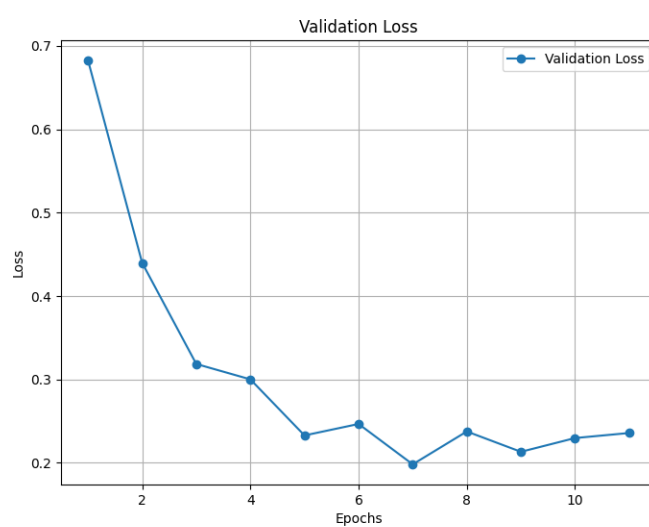


Figure 4.11

Chapter 5

Final evaluation

5.1 Video Test

Bounding boxes are used to crop images from the original frame, which, after transformations (the same as those used during training, with the addition of CLAHE), are passed to the classification model.

To avoid frequent label changes on the video, we implemented a moving average over a specific time window. This approach significantly improves the stability of results displayed on the video.

In the final result processing, a prediction is considered positive (1) if the count of positive predictions throughout the video's history is greater than the count of negative predictions (and vice versa).

The frame processing, from the perspective of the tracker and detector, takes an average of 50ms, even when skipping 7 frames. The code for generating bounding boxes takes approximately 20 milliseconds to execute and runs in parallel with the detecting and tracking operations.

Classification is the most computationally expensive process; for a single frame, it takes about 200ms on average (obviously depending on the number of objects to classify in that frame).

In total, the processing takes about 260 every frame, which means that for real-time processing, we need to process a maximum of $\frac{1000}{250} = 4$ frames per second.

Attempting to skip 15 frames per second (by setting the `vid_stride` parameter) resulted in a video that was not smooth, and the outcomes of the tracking and detecting tasks were unsatisfactory. Consequently, it was decided to apply classification every 4 frames to meet the real-time constraint, setting the `vid_stride` value to 7, which was also the value chosen for the detector.

Classification occurs every $4 \times 7 = 28$ frames, meaning it is performed twice per second (60/28), while the video is processed 8 times per second (60/7) by the tracker.

The following result shows the total amount of ms needed for the execution:

$$4 * 70 + 200 = 480$$

The four skipped frames each require 70 ms of execution time, while classification takes 200 ms. This brings the total processing time for a second (for a video of 60 fps) to $480 \times 2 = 960$ ms, a value that allows real-time processing.

To improve the prediction, we also used the CLAHE transformation, which led to a noticeable improvement in the prediction of "bag" as well as increased robustness for the other two labels.

Of the four videos used, two were utilized for the optimal threshold values to balance false positives (FP) and false negatives (FN) and two (recorded in the same conditions) for actual testing. The tested threshold values and the number of errors found are listed below. The attempts made to obtain the parameters are as follows:

Table 5.1: Threshold Results

Configuration	T Gender	T Bag	T Hat	Total Errors (Type)
1	0.6	0.5	0.5	4 (1 Gender, 3 Bag)
2	0.6	0.3	0.5	3 (1 Gender, 2 Bag)
3	0.5	0.3	0.5	–

The final parameters tested were those ultimately chosen for video testing :

Table 5.2: Detected Errors

Video	Total Errors (Type)
Originale	2
Simile	1
Testing 1	0
Testing 2	3 (2 Gender, 1 Hat)

The tracker and detector correctly track people thanks to the chosen configuration. The classifier performs well on the test videos for gender and bag across various angles; however, it struggles with hat classification, contrary to our expectations based on the evaluated F1-score.

5.2 Possible Improvements

The test set was not as useful as expected. In fact, almost all the tested models showed good performance on the test set (derived from the provided validation set), with F1-scores ranging from a minimum of 0.7 to a maximum of 0.92. However, the results on the video were significantly different.

We deduce that the test set is insufficient for evaluating the classifier in a real-world environment. It would likely be beneficial to make the video as similar as possible to the training images.

The results show good F1-scores, particularly for gender and hat, while bag proved to be the most challenging task with the lowest F1-score. If more time and data were available to train the multi-task model, the classification quality could potentially improve.

Further exploration of GradNorm could have led to better results, or the use of potentially more powerful backbones

To compensate for the anomalous distribution of the training set, an alternative to adjusting the loss could have been the implementation of data augmentation. However, this approach was not pursued due to concerns about inadvertently introducing additional biases into the dataset.

A challenge to address is that often the bounding boxes also include part of the nearby bounding box (in the case of people standing next to each other), which causes the classifier to struggle with the prediction. However, the use of the moving average helps recover from these errors. If there had been more time and resources, a connected components algorithm could have been implemented to recognize each person and classify them individually.